

Boiler Controller Console Application

SECTION I: General Requirements

The application should do the following:

- **Function as specified in Section III of this document.**
- **Adhere to C# Coding Conventions:** Conform to the coding style and documentation standards as outlined in coding conventions.
- **Be developed expressly for this exam:** All code should be written specifically for this assessment. Utilizing templates, examples, or libraries developed outside the bounds of this exam is prohibited, unless explicitly allowed.
- **Implement proper error handling:** Ensure that the application gracefully handles unexpected situations or errors, using mechanisms like try-catch blocks and custom exception classes.
- **Maintain modularity:** All major functionalities should be encapsulated within methods or separate classes. Avoid long monolithic methods; instead, break down the code into smaller, purpose-specific methods.
- **Ensure scalability:** The design and code structure should allow for easy addition of new features or functionalities without significant restructuring.
- **Thorough documentation:** This includes:
 - Comments explaining the purpose and functionality of sections of code.
 - Documentation for classes, methods, and properties.
 - Proper indentation and code organization for readability.
 - Meaningful naming conventions for classes, methods, variables, and namespaces.
 - **Input validation:** Ensure that the application adequately validates and sanitizes user input to prevent errors and potential security vulnerabilities.
- **Logging and feedback:** The console application should provide meaningful feedback to the user about the operations it's performing, potential errors, and their causes. Consider implementing logging mechanisms to capture important events or errors.

SECTION II: Evaluation Parameters

S. No	Criteria/Parameter	Total Score
01	Comprehending the requirements	020
02	Proper understanding and usage of technical concepts	020
03	Quality of Solution - Functionality & Corner Cases	020
04	Quality of Code - Style, Standards & Documentation	020
05	Breakdown and Prioritization	010
06	UI Customization	010
	TOTAL	100

Section III: Boiler

Objective

Design and implement a comprehensive boiler startup controller using C#. The application will simulate the operations of a boiler system, reflecting real-world scenarios with detailed state transitions, timers, and logging capabilities.

Add proper error handling.

Requirements

1. **Application Initialization:**
 - As the application starts, display a welcoming message: "Boiler Controller Initialized."
 - Initialize the boiler's system status to "Lockout."
 - Initiate the event log, capturing all significant events. The first log entry should be "Boiler Initialized" with the current timestamp. (Refer to "Event Log File Specification" for complete details)
2. **Console Interface Presentation:**
 - Present a clear menu to the user, making it intuitive to navigate between operations:
 - Start Boiler Sequence
 - Stop Boiler Sequence
 - Simulate Boiler Error. (Can be used to Simulate error only when the boiler is in Operational mode)
 - Toggle Run Interlock Switch (Open/Closed)
 - Reset Lockout
 - View Event Log
 - Exit Application
3. **Run Interlock Simulation:**
 - This is a safety feature. At initialization, the switch is in the "Open" position.
 - The user must be able to toggle this switch.
 - The boiler can only be ready or started if the switch is in the "Closed" position.
 - Each time the switch state changes, log the event with a message like "Interlock Switch toggled to [state]."
4. **Lockout Reset Functionality:**
 - Reset is essential after any failure or after initialization.
 - If the "Run Interlock" switch is in the "Closed" position, the system status should transition to "Ready." Else, inform the user of the need to close the interlock switch.
 - Log the event: "Boiler Status changed to Ready" with the timestamp.
5. **Boiler Start Sequence:**
 - This sequence simulates the various phases of a boiler startup.
 - **Pre-Purge Cycle:**
 - Lasts for 10 seconds.
 - Status: "Pre-Purge"
 - Users should see a countdown or elapsed time.
 - At the end, log: "Pre-Purge completed."
 - **Ignition Phase:**
 - Lasts for 10 seconds.
 - Status: "Ignition"

- Similarly, provide a time indication for the user.
 - Log: "Ignition phase completed."
 - **Operational State:**
 - After ignition, transition to "Operational."
 - Log: "Boiler now operational." Continue to be in this state.
6. **Event Logging Mechanism:**
- Capture all significant events in an event log (A file on disk).
 - Each log entry should have a timestamp, event name, and relevant data (if any).
 - Provide an option for the user to view this log, which should display the entries in a clear, readable format.
7. **Safety and Error Handling:**
- If there is any simulated failure or error during a phase, transition back to the "Lockout" state.
 - Display a clear error message: "Error: [Error Description]. System in Lockout."
 - Log the error with its timestamp.
8. **Non-Functional Requirements:**
- Handle any invalid user inputs gracefully, providing guidance on the correct action.
 - After each operation or error, loop back to the main menu, ensuring the user can perform multiple actions seamlessly.
 - Ensure code is modular, making it easy to potentially add more phases or features in the future.

Event Log File Specification

- File Name: Boiler Log.txt
- File Location: Parallel to Exe.
- Format: Comma Separated Value (.csv)
- File Header: Timestamp, Event, Event Data
- Event Data string format: Absolute date and time string, Event string, Event data string
- File Creation and Modification: If the file does not exist, create new one, write header and log event data. If file exists, append event data.

Happy Coding!